

21CSC558J - DEEP LEARNING APPROACHES

Module-2 - Optimization, Regularization and Normalization in Deep Neural Networks

Session-3

Gradient Descent Method

- Gradient descent is a way to minimize an objective function $J(\theta)$
 - $\theta \in \mathbb{R}^d$: model parameters
 - η : learning rate
 - $\nabla_{\theta} J(\theta)$: gradient of the objective function with regard to the parameters
- Updates parameters **in opposite direction** of gradient.
- Update equation: $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$

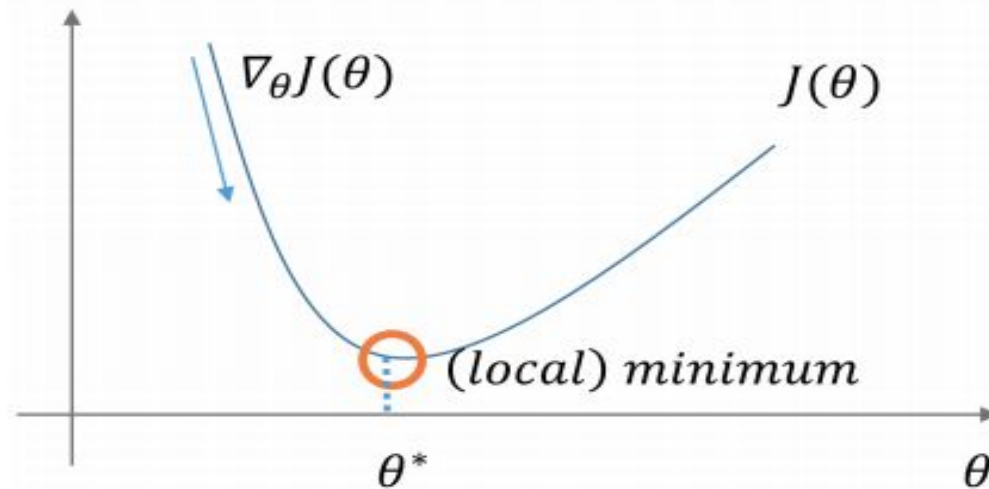


Figure: Optimization with gradient descent

Gradient Descent Variants

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-batch Gradient Descent
- Difference: how much data we use in computing the *gradients*

Types of Gradient Descent

Batch Gradient Descent (BGD)

- Uses the **entire dataset** to compute the gradient.
- Converges **smoothly** but can be **slow for large datasets**.

Stochastic Gradient Descent (SGD)

- Updates parameters using a **single random data point per iteration**.
- **Faster** updates but introduces **high variance**, leading to **noisy convergence**.

Mini-Batch Gradient Descent

- Uses a **small batch** of data to compute the gradient.
- Balances between BGD and SGD, providing **efficiency and stability**.

Reasons

Minimizing Loss Functions

- Machine learning models learn by minimizing a loss function (e.g., Mean Squared Error, Cross-Entropy Loss).
- Gradient Descent helps find the **best parameters** (weights and biases) to reduce this loss.

Efficient Optimization in High Dimensions

- Modern deep learning models have **millions of parameters**.
- Analytical solutions (like solving derivatives directly) become impractical.
- Gradient Descent allows iterative updates without needing to compute **complex derivatives manually**.

Reasons

Works for Non-Linear and Complex Models

- **Many real-world problems** involve non-linear, high-dimensional data.
- Gradient Descent can optimize models like neural networks, which don't have closed-form solutions.

Scalability for Large Datasets

- Variants like **Stochastic Gradient Descent (SGD)** and **Mini-Batch Gradient Descent** allow efficient learning from massive datasets.

Avoids Exhaustive Search

- Instead of testing **all possible parameter values** (which is computationally impossible), Gradient Descent efficiently follows the negative gradient to reach an optimal solution.

Advantage

Efficient for Large Datasets

- **Stochastic and Mini-Batch Gradient Descent** allow training on large datasets efficiently.

Scalability to High Dimensions

- Works well even when the number of parameters (e.g., in deep learning) is in the **millions**.

Generalization to Complex Models

- Can optimize both **linear and non-linear models**, including neural networks.

Flexible Optimization Variants

- Optimizers like **Momentum, RMSProp, and Adam** improve convergence speed and stability.

Guaranteed Convergence for Convex Functions

- If the function is convex and the learning rate is set properly, it will always find the **global minimum**.

Disadvantage

Sensitive to Learning Rate

- Too high: May **oscillate** or diverge.
- Too low: Converges very **slowly**.

Prone to Local Minima in Non-Convex Functions

- In deep learning, the loss function is non-convex, leading to **local minima** or saddle points.

Computationally Expensive for Full-Batch Gradient Descent

- Computing gradients over the entire dataset in each iteration can be **slow**.

Requires Feature Scaling

- **Without proper normalization**, training can be slow and inefficient.

Noise in Stochastic Gradient Descent (SGD)

- While SGD is fast, its updates **introduce randomness**, causing instability in convergence.

AdaGrad – Definition and Properties

Adagrad (Adaptive Gradient Algorithm) is an optimization algorithm that adapts the learning rate for each parameter based on past gradients.

It assigns smaller learning rates to frequently updated parameters and larger learning rates to rarely updated ones.

Properties

(i) Adaptive Learning Rate for Each Parameter

- Unlike SGD, which uses a fixed learning rate for all parameters, Adagrad **adjusts the learning rate for each parameter individually**.
- Parameters with **frequent updates** get a **smaller** learning rate.
- Parameters with **rare updates** get a **larger** learning rate.

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

where $G_t = G_{t-1} + g_t^2$ accumulates past gradients.

This file is meant for personal use by ravikanthamajala301@hotmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

AdaGrad - Properties

(ii) Accumulation of Squared Gradients

The denominator $\sqrt{G_t}$ keeps increasing over time.

- This **reduces the learning rate** for frequently updated parameters.

Effect:

- **Good for convex problems** (e.g., logistic regression, SVM).
- Bad for deep learning → Learning rate **shrinks too much**, causing premature convergence.

(iii) Works Well with Sparse Data

- Adagrad helps models train on **sparse features**, common in **Natural Language Processing (NLP) and recommendation systems**.
- Example: If a word appears rarely in text data, Adagrad ensures it gets a relatively higher learning rate.

AdaGrad - Properties

(iv) No Manual Learning Rate Decay Needed

- Unlike SGD, Adagrad **automatically adjusts** learning rates over time.
- No need for manually tuning **learning rate schedules**.

(v) Limitation: Learning Rate Decay

- The sum of squared gradients (**Gt**) **keeps growing**, making the learning rate **too small over time**.
- Solution: **RMSprop** and **Adam** fix this by using an exponentially moving average instead of accumulating all past gradients.

AdaGrad - Algorithm

Let:

- $\eta \rightarrow$ Initial learning rate
- $g_t \rightarrow$ Gradient at time t
- $G_t \rightarrow$ Sum of past squared gradients
- $\theta_t \rightarrow$ Parameter to be updated
- $\epsilon \rightarrow$ Small constant to prevent division by zero

Update Rule

1. Compute the sum of squared past gradients:

$$G_t = G_{t-1} + g_t^2$$

2. Update the parameter:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

AdaGrad - Example

Adagrad **accumulates squared gradients**, making the denominator grow continuously.

Initial parameter $\theta_0 = 1.5$

Initial accumulated squared gradient $G_0 = 0$ (for Adagrad)

Learning rate $\eta = 0.01$

Smoothing term $\epsilon = 10^{-8}$

Decay factor $\gamma = 0.9$ (for RMSprop)

Given gradients over 3 iterations:

- $g_1 = 0.8$
- $g_2 = 0.6$
- $g_3 = 0.4$

Iteration 1

$$G_1 = G_0 + g_1^2 = 0 + (0.8)^2 = 0.64$$

$$\theta_1 = \theta_0 - \frac{\eta}{\sqrt{G_1 + \epsilon}} g_1$$

Approximating:

$$\frac{0.01}{\sqrt{0.64 + 10^{-8}}} \approx \frac{0.01}{0.8} = 0.0125$$

$$\theta_1 = 1.5 - (0.0125 \times 0.8) = 1.49$$

AdaGrad - Example

Iteration 2

$$G_2 = G_1 + g_2^2 = 0.64 + (0.6)^2 = 1.00$$

$$\frac{0.01}{\sqrt{1.00 + \epsilon}} \approx \frac{0.01}{1} = 0.01$$

$$\theta_2 = 1.49 - (0.01 \times 0.6) = 1.484$$

Iteration 3

$$G_3 = G_2 + g_3^2 = 1.00 + (0.4)^2 = 1.16$$

$$\frac{0.01}{\sqrt{1.16 + \epsilon}} \approx \frac{0.01}{1.077} \approx 0.0093$$

$$\theta_3 = 1.484 - (0.0093 \times 0.4) = 1.4803$$

◆ **Observation:** The denominator $\sqrt{G_t}$ keeps increasing, making step sizes smaller and smaller.

Learning rate shrinks too much over time.

RMSprop – Definition and Purpose

RMSprop (Root Mean Square Propagation) is an adaptive optimization algorithm that improves the stability and efficiency of gradient descent.

It is designed to **control learning rate fluctuations, accelerate convergence, and handle non-stationary loss landscapes** effectively.

Purpose

(i) **Adaptive Learning Rate Scaling**

In standard Stochastic Gradient Descent (SGD), a fixed learning rate is applied to all parameters, which may lead to slow convergence or divergence if not tuned properly.

RMSprop overcomes this issue by adjusting the learning rate dynamically for each parameter based on the recent magnitude of the gradients.

Purpose: This prevents drastic changes in parameter updates and ensures smoother optimization.

Effect: Smaller learning rates for frequently updated parameters and larger learning rates for infrequent updates.

RMSprop – Definition and Purpose

(ii) Prevents Oscillations and Stabilizes Training

When optimizing deep neural networks, especially on highly non-convex loss surfaces, large gradients in some directions cause erratic updates, leading to instability.

RMSprop smooths out gradient updates by maintaining a moving average of squared gradients.

Purpose: Reduces the risk of oscillations by normalizing the step size for each parameter.

Effect: Allows faster convergence without diverging, especially in deep learning models.

(iii) Efficient Handling of Non-Stationary Objectives

In deep learning, loss landscapes often change as training progresses.

RMSprop dynamically adjusts the step size for each parameter based on recent gradient behavior.

Purpose: Helps optimize dynamic loss functions that evolve over training.

Effect: More effective learning across different epochs and changing gradients.

RMSprop – Definition and Purpose

(iv) Prevents Vanishing/Exploding Gradients

When training deep networks, gradients may become too small (vanishing gradients) or too large (exploding gradients).

RMSprop mitigates this by normalizing gradients using their recent squared values.

Purpose: Ensures stable weight updates and prevents extreme gradient variations.

Effect: Makes deep networks trainable without excessive gradient clipping.

(v) Faster Convergence in Deep Learning Models

By adaptively controlling the learning rate, RMSprop allows faster training compared to SGD, especially for deep architectures such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

Purpose: Reduces the number of training epochs required for convergence.

Effect: Efficiently trains deep learning models without fine-tuning the learning rate manually.

RMSprop – Definition and Purpose

(vi) Handles Sparse Data More Effectively

In scenarios where gradients are sparse (e.g., NLP tasks), traditional optimizers struggle to balance weight updates across different features.

RMSprop effectively scales learning rates per parameter, making it suitable for sparse datasets.

Purpose: Ensures even rare features are updated meaningfully.

Effect: Improved learning in sparse data applications like text processing.

RMSprop – Algorithm

1. Initialize parameters:

- Set $\theta = 1.5$
- Set $E[g^2] = 0$

2. For each iteration t from 1 to T :

a. Compute the moving average of squared gradients:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

b. Compute the updated parameter:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

3. Repeat until convergence or until T iterations are completed.

4. Output final θ value.

RMSprop – Example

Initial parameter: $\theta_0 = 1.5$

Initial accumulated squared gradient: $E[g^2]_0 = 0$

Learning rate: $\eta = 0.01$

Smoothing term: $\epsilon = 10^{-8}$

Decay factor: $\gamma = 0.9$

Gradients: $g_1 = 0.8, g_2 = 0.6, g_3 = 0.4$

Iteration 1:

- Given: $\theta_0 = 1.5, E[g^2]_0 = 0, g_1 = 0.8$
- Compute moving average:

$$E[g^2]_1 = (0.9 \times 0) + (0.1 \times 0.8^2)$$

$$E[g^2]_1 = 0 + 0.1 \times 0.64 = 0.064$$

- Update parameter:

$$\theta_1 = 1.5 - \frac{0.01}{\sqrt{0.064 + 10^{-8}}} \times 0.8$$

$$\theta_1 = 1.5 - \frac{0.01}{0.252} \times 0.8$$

$$\theta_1 = 1.5 - (0.03968)$$

$$\theta_1 \approx 1.4603$$

RMSprop – Example

Iteration 2:

- Given: $\theta_1 = 1.4603$, $E[g^2]_1 = 0.064$, $g_2 = 0.6$
- Compute moving average:

$$E[g^2]_2 = (0.9 \times 0.064) + (0.1 \times 0.6^2)$$

$$E[g^2]_2 = 0.0576 + 0.1 \times 0.36$$

$$E[g^2]_2 = 0.0576 + 0.036 = 0.0936$$

- Update parameter:

$$\theta_2 = 1.4603 - \frac{0.01}{\sqrt{0.0936 + 10^{-8}}} \times 0.6$$

$$\theta_2 = 1.4603 - \frac{0.01}{0.306} \times 0.6$$

$$\theta_2 = 1.4603 - (0.0196)$$

$$\theta_2 \approx 1.4407$$

RMSprop – Example

Iteration 3:

- Given: $\theta_2 = 1.4407$, $E[g^2]_2 = 0.0936$, $g_3 = 0.4$
- Compute moving average:

$$E[g^2]_3 = (0.9 \times 0.0936) + (0.1 \times 0.4^2)$$

$$E[g^2]_3 = 0.08424 + 0.1 \times 0.16$$

$$E[g^2]_3 = 0.08424 + 0.016 = 0.10024$$

- Update parameter:

$$\theta_3 = 1.4407 - \frac{0.01}{\sqrt{0.10024 + 10^{-8}}} \times 0.4$$

$$\theta_3 = 1.4407 - \frac{0.01}{0.3167} \times 0.4$$

$$\theta_3 = 1.4407 - (0.01263)$$

$$\theta_3 \approx 1.4281$$

◆ **Observation:** The denominator remains controlled and does not grow indefinitely. **Step sizes remain stable, preventing premature convergence.**

RMSprop – Example

Iteration 3:

- Given: $\theta_2 = 1.4407$, $E[g^2]_2 = 0.0936$, $g_3 = 0.4$
- Compute moving average:

$$E[g^2]_3 = (0.9 \times 0.0936) + (0.1 \times 0.4^2)$$

$$E[g^2]_3 = 0.08424 + 0.1 \times 0.16$$

$$E[g^2]_3 = 0.08424 + 0.016 = 0.10024$$

- Update parameter:

$$\theta_3 = 1.4407 - \frac{0.01}{\sqrt{0.10024 + 10^{-8}}} \times 0.4$$

$$\theta_3 = 1.4407 - \frac{0.01}{0.3167} \times 0.4$$

$$\theta_3 = 1.4407 - (0.01263)$$

$$\theta_3 \approx 1.4281$$

◆ **Observation:** The denominator remains controlled and does not grow indefinitely. **Step sizes remain stable, preventing premature convergence.**

ADAM – Definition and Purpose

Adam (Adaptive Moment Estimation) is an optimization algorithm that combines Momentum and RMSprop, adjusting learning rates dynamically for each parameter.

It provides faster convergence and better stability, making it ideal for deep learning applications.

Purpose

(i) Adaptive Learning Rate

Purpose : Adam dynamically adjusts the learning rate for each parameter based on past gradients, rather than using a fixed learning rate like standard SGD.

Effect: This prevents overshooting in high-gradient areas and vanishing updates in low-gradient regions, leading to more stable and efficient learning.

(ii) Combining Momentum & RMSprop

Purpose: Momentum helps smooth updates by keeping track of past gradients, while RMSprop adapts the learning rate based on the magnitude of past squared gradients.

Effect: By leveraging both methods, Adam prevents abrupt changes in weight updates, improving convergence speed and reducing oscillations during training.

ADAM – Definition and Purpose

(iii) Faster Convergence

Purpose: Adam optimizes parameter updates by using adaptive learning rates and momentum, reducing unnecessary computations and making updates more efficient.

Effect: This significantly shortens training time, making deep learning models more practical and scalable, especially when dealing with large datasets.

(iv) Better Stability

Purpose: Adam is designed to handle noisy and sparse gradients, making it well-suited for problems with irregular updates, such as NLP and sparse data applications.

Effect: The optimizer ensures smoother and more reliable training, preventing fluctuations that could lead to suboptimal solutions or slow progress.

ADAM – Definition and Purpose

(v) Bias Correction for Early Steps

Purpose: In the early iterations, the moment estimates (first and second moments) can be biased toward zero due to their dependence on initial conditions.

Adam applies bias correction to counteract this issue.

Effect: This leads to more accurate gradient updates at the start of training, allowing the optimizer to settle into an optimal learning path more quickly.

ADAM – Definition and Purpose

(v) Bias Correction for Early Steps

Purpose: In the early iterations, the moment estimates (first and second moments) can be biased toward zero due to their dependence on initial conditions.

Adam applies bias correction to counteract this issue.

Effect: This leads to more accurate gradient updates at the start of training, allowing the optimizer to settle into an optimal learning path more quickly.

ADAM – Algorithm

Let:

- $\eta \rightarrow$ Learning rate (default: 0.001)
- $\beta_1 \rightarrow$ Decay rate for **momentum** (default: 0.9)
- $\beta_2 \rightarrow$ Decay rate for **RMSprop-like variance** (default: 0.999)
- $\epsilon \rightarrow$ Small constant to avoid division by zero (default: 10^{-8})
- $g_t \rightarrow$ Gradient at time step t
- $m_t \rightarrow$ First moment estimate (momentum term)
- $v_t \rightarrow$ Second moment estimate (variance term)
- $\theta_t \rightarrow$ Parameter to be updated

ADAM – Algorithm

1. **Initialize** parameters θ_0 , moment estimates $m_0 = 0, v_0 = 0$.
2. **Compute** gradients g_t at time step t .
3. **Update** biased first moment estimate (momentum-like behavior):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

4. **Update** biased second moment estimate (RMSprop-like behavior):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

5. **Bias correction** for moments:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

These corrections **prevent initial bias** at the beginning of training.

ADAM – Algorithm

6. Update parameters:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

7. Repeat until convergence.

ADAM – Example

- Initial Parameter: $\theta_0 = 1.5$
- Learning Rate: $\eta = 0.01$
- Smoothing Term: $\epsilon = 10^{-8}$
- Decay Factors:
 - $\beta_1 = 0.9$ (for momentum)
 - $\beta_2 = 0.999$ (for variance scaling)
- Gradients:
 - $g_1 = 0.8$
 - $g_2 = 0.6$
 - $g_3 = 0.4$

ADAM – Example

Iteration 1 ($t = 1$)

1. Momentum update

$$m_1 = \beta_1 \cdot m_0 + (1 - \beta_1) \cdot g_1$$

$$m_1 = (0.9 \times 0) + (0.1 \times 0.8) = 0.08$$

2. Variance update

$$v_1 = \beta_2 \cdot v_0 + (1 - \beta_2) \cdot g_1^2$$

$$v_1 = (0.999 \times 0) + (0.001 \times 0.8^2) = 0.00064$$

3. Bias correction

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{0.08}{1 - 0.9} = 0.8$$

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^1} = \frac{0.00064}{1 - 0.999} = 0.64$$

ADAM – Example

4. Parameter update

$$\theta_1 = \theta_0 - \frac{\eta}{\sqrt{\hat{v}_1} + \epsilon} \hat{m}_1$$

$$\theta_1 = 1.5 - \frac{0.01}{\sqrt{0.64} + 10^{-8}} \times 0.8$$

$$\theta_1 = 1.5 - \frac{0.01}{0.8} \times 0.8$$

$$\theta_1 = 1.5 - 0.01 = 1.49$$

Iteration 2 ($t = 2$)

1. Momentum update

$$m_2 = (0.9 \times 0.08) + (0.1 \times 0.6) = 0.128$$

2. Variance update

$$v_2 = (0.999 \times 0.00064) + (0.001 \times 0.6^2) = 0.000976$$

ADAM – Example

3. Bias correction

$$\hat{m}_2 = \frac{m_2}{1 - \beta_1^2} = \frac{0.128}{1 - 0.9^2} = 0.6737$$

$$\hat{v}_2 = \frac{v_2}{1 - \beta_2^2} = \frac{0.000976}{1 - 0.999^2} = 0.488$$

4. Parameter update

$$\theta_2 = 1.49 - \frac{0.01}{\sqrt{0.488 + 10^{-8}}} \times 0.6737$$

$$\theta_2 = 1.49 - \frac{0.01}{0.698} \times 0.6737$$

$$\theta_2 = 1.49 - 0.00965 = 1.4804$$

ADAM – Example

Iteration 3 ($t = 3$)

1. Momentum update

$$m_3 = (0.9 \times 0.128) + (0.1 \times 0.4) = 0.1552$$

2. Variance update

$$v_3 = (0.999 \times 0.000976) + (0.001 \times 0.4^2) = 0.001136$$

3. Bias correction

$$\hat{m}_3 = \frac{m_3}{1 - \beta_1^3} = \frac{0.1552}{1 - 0.9^3} = 0.613$$

$$\hat{v}_3 = \frac{v_3}{1 - \beta_2^3} = \frac{0.001136}{1 - 0.999^3} = 0.379$$

ADAM – Example

4. Parameter update

$$\theta_3 = 1.4804 - \frac{0.01}{\sqrt{0.379} + 10^{-8}} \times 0.613$$

$$\theta_3 = 1.4804 - \frac{0.01}{0.615} \times 0.613$$

$$\theta_3 = 1.4804 - 0.00996 = 1.4704$$

Momentum smooths updates → mt accumulates past gradients gradually.

Variance prevents drastic updates → vt scales learning rates dynamically.

Adaptive learning rate prevents vanishing steps compared to Adagrad.

Unit Saturation – Definition and Types

Unit saturation occurs in neural networks when a neuron's activation function outputs values that are extremely close to its upper or lower limits.

This makes gradients very small (or even zero), preventing weight updates and slowing down learning, leading to the vanishing gradient problem.

Types

(i) Sigmoid Saturation

Occurs when: The input to the sigmoid function is too large (positive or negative).

Effect: Outputs are close to 0 or 1, making gradients approach zero.

Solution: Use ReLU instead of sigmoid for deep networks.

Unit Saturation – Types

(ii) Tanh Saturation

Occurs when: The input to the tanh function is too large.

Effect: Outputs are close to -1 or 1, leading to small gradients.

Solution: Weight initialization techniques like Xavier initialization help mitigate this.

(iii) ReLU Dead Neurons (Dying ReLU)

Occurs when: The input to the ReLU function is negative, always producing 0.

Effect: Neurons stop updating, reducing network capacity.

Solution: Use Leaky ReLU or ELU to avoid dying neurons.

Unit Saturation – Types

(ii) Tanh Saturation

Occurs when: The input to the tanh function is too large.

Effect: Outputs are close to -1 or 1, leading to small gradients.

Solution: Weight initialization techniques like Xavier initialization help mitigate this.

(iii) ReLU Dead Neurons (Dying ReLU)

Occurs when: The input to the ReLU function is negative, always producing 0.

Effect: Neurons stop updating, reducing network capacity.

Solution: Use Leaky ReLU or ELU to avoid dying neurons.

Unit Saturation – Features and Purpose

Features

Leads to vanishing gradients: Small gradient updates slow down learning.

Common in deep networks: More layers increase the risk of saturation.

Depends on activation functions: Sigmoid, tanh, and ReLU can all experience saturation.

Affects weight updates: Causes slow convergence or stops training progress.

Purpose

Prevents Training Issues: Identifying saturation early helps fix vanishing gradients.

Improves Model Performance: Avoiding saturation ensures smooth gradient flow.

Guides Activation Function Choice: Helps in selecting appropriate functions like ReLU, Leaky ReLU, or Swish.

Unit Saturation – Advantage

Advantages of Unit Saturation

- Helps Control Output Range In functions like sigmoid and tanh, saturation ensures outputs remain within a fixed range (e.g., $[0,1]$ or $[-1,1]$), preventing extremely large values.

Useful for Probability-Based Outputs

- The sigmoid function is commonly used in binary classification problems because its output can be interpreted as probabilities.

Stable Representations in Shallow Networks

- In simple models, unit saturation can sometimes help stabilize representations by reducing drastic weight updates.

Unit Saturation – Disadvantage

Causes Vanishing Gradient Problem

- When neurons saturate, gradients become very small, slowing down or stopping learning in deep networks.

Slows Convergence

- Saturated units result in very small weight updates, leading to slow training and inefficient learning.

Leads to Dead Neurons (in ReLU)

- In ReLU, neurons can "die" if they always output zero due to negative inputs, making them inactive forever.

Not Suitable for Deep Networks

- Activation functions prone to saturation (sigmoid, tanh) struggle in deep neural networks, requiring careful initialization and optimization techniques.

Vanishing Gradient – Definition

The vanishing gradient problem occurs when gradients become extremely small during backpropagation in deep neural networks.

This leads to **insignificant weight updates**, slowing down learning and sometimes stopping it completely.

Side Effects

Slow or No Learning

- If gradients shrink too much, weight updates become negligible, preventing the model from learning effectively.

Poor Performance in Deep Networks

- Deep networks require significant weight updates to learn complex patterns.
- Vanishing gradients **prevent lower layers from learning, leading to underfitting.**

Vanishing Gradient – Side Effects

Longer Training Time

- Small gradients mean small updates, requiring more epochs to converge.

Difficulty in Training Recurrent Neural Networks (RNNs)

- In RNNs, vanishing gradients cause older time steps to have almost no influence on the output, making long-term dependencies difficult to learn.

Vanishing Gradient – Reason

Suppose we have a deep network with multiple layers.

During backpropagation, the gradient of each layer is computed as

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial W_i}$$

where L is the loss, W_i are weights, and a_i is the activation.

If the activation function is sigmoid, its derivative is

$$f'(x) = \sigma(x)(1 - \sigma(x))$$

The maximum value of $f'(x)$ is 0.25, meaning gradients shrink at every layer.

If we multiply many small derivatives across layers, they become close to zero

$$\text{Gradient} = 0.25 \times 0.25 \times 0.25 \times \dots \approx 0$$

leading to vanishing gradients.

Vanishing Gradient – Example

Let's consider a 3-layer neural network using the sigmoid activation function and compute gradients step by step.

Initial gradients: $g_3 = 0.8$

Sigmoid derivative at each layer: $f'(x) = 0.25$

Backpropagation Step:

$$g_2 = g_3 \times 0.25 = 0.8 \times 0.25 = 0.2$$

$$g_1 = g_2 \times 0.25 = 0.2 \times 0.25 = 0.05$$

$$g_0 = g_1 \times 0.25 = 0.05 \times 0.25 = 0.0125$$

After just 3 layers, the gradient has shrunk from **0.8** to **0.0125**, making learning extremely slow.

In deeper networks, this effect is even more pronounced, making early layers almost **untrainable**.

The vanishing gradient problem significantly affects deep networks, preventing effective learning.

ReLU, batch normalization, residual connections, and better initialization help mitigate this issue.

Exploding Gradients – Definition & Side Effects



The exploding gradient problem occurs when gradients grow exponentially during backpropagation in deep neural networks.

This causes unstable weight updates, making the model diverge instead of learning properly.

Side Effects

Unstable Training

- The model's loss fluctuates wildly and does not converge to an optimal value.

Divergence of Weights

- Large gradients lead to excessive weight updates, making the model unstable.

Nan or Inf Loss Values

- Extreme weight values cause numerical overflow, leading to NaN (Not a Number) or Inf (Infinity) values in loss calculations.

Performance Issues in RNNs

- In recurrent neural networks (RNNs), exploding gradients prevent long-term dependency learning and cause erratic updates.

Exploding Gradients – Reason

1. Consider a deep network with multiple layers.
2. During backpropagation, the gradient update is computed as

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial W_i}$$

3. If the weight gradients are greater than 1, multiplying them across layers causes exponential growth

$$\text{Gradient} = 2 \times 2 \times 2 \times \dots = 2^n$$

where n is the number of layers.

4. This results in huge weight updates, leading to instability.

Exploding Gradients – Example

Let's consider a 3-layer neural network with a gradient greater than 1.

Initial gradients: $g_3 = 2.5$

Weight derivatives at each layer: $f'(x) = 2$

Backpropagation Step

$$g_2 = g_3 \times 2 = 2.5 \times 2 = 5$$

$$g_1 = g_2 \times 2 = 5 \times 2 = 10$$

$$g_0 = g_1 \times 2 = 10 \times 2 = 20$$

After just 3 layers, the gradient has exploded from 2.5 to 20.

In deeper networks, this effect can quickly make gradients too large, destabilizing training.

The exploding gradient problem can cause unstable training and divergence.

Gradient clipping, weight regularization, batch normalization, and proper initialization help mitigate this issue.

Exploding Gradients – Solutions

Gradient Clipping

- Limits gradient values to a predefined threshold

$$g = \frac{g}{\max(1, \frac{|g|}{t})}$$

where t is the clipping threshold.

Use Smaller Learning Rates

- Reducing the learning rate prevents drastic weight updates.

Weight Regularization (L2 Regularization)

- Adds a penalty to large weight values, controlling their magnitude.

Use Proper Weight Initialization

- Xavier or He initialization prevents large initial weight values that could lead to exploding gradients.

Batch Normalization

- Normalizes activations to keep values within a reasonable range, reducing extreme gradients.